SPRINT 01, FUNDAMENTOS DO PROCESSO

Métricas de processo de software

Se você não mede, está chutando. Hoje a gente aprende a medir com menos teatro e mais consequência: DORA, SLOs, observabilidade, e as armadilhas clássicas.

Aula	03 de 14
Data	28 de abril de 2026, terça-feira, 10h00 às 12h00
Professor	José Romualdo da Costa Filho
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

A aula é quase toda em sala de aula invertida. Pouco slide expositivo, muita discussão em dupla, em grupo e em plenário. Vocês vão ler, interpretar, criar e criticar métricas. No papel e com voz.

Bloco	Atividade	Duração
10h00, Daily Time	Abertura com cronômetro de 15 min, status rápido por grupo	15 min
10h15, Ativação	Três quizzes sobre o auto-estudo, DORA, vanity e observabilidade	15 min
10h30, Síntese	Consolidar DORA, SLI/SLO/SLA e os três pilares	15 min
10h45, Interpretação	Três FLIPs em dupla, ler painéis, achar Goodhart, escolher pilar	30 min
11h15, Criação	Cada grupo projeta três métricas para o próprio projeto	15 min
11h30, Defesa cruzada	Pitch de 1 minuto por grupo, crítica obrigatória dos pares	15 min
11h45, Síntese	Goodhart e Campbell, ponte TAP Jacto e auto-estudo	15 min

DAILY, 15 MIN

Daily Time

Status de cada grupo: o que avançou desde ontem, o que trava, qual é a próxima meta. Objetivo, não narração.

15:00

INICIAR PAUSAR ZERAR

Objetivos de aprendizagem

inteli.

A aula gira em torno de quatro verbos. A ideia não é decorar definições, é sair daqui sabendo ler um painel, montar um conjunto de métricas para o seu projeto e desconfiar do que você acabou de montar.

Interpretar

Ler um painel DORA e não confundir alta atividade com alto resultado.

Comparar

Distinguir DORA, SLI/SLO/SLA e métricas de qualidade, cada uma no seu lugar.

Criar

Projetar um conjunto mínimo de métricas para o seu projeto, com fórmula e fonte.

Criticar

Aplicar Goodhart, Campbell e o teste de vaidade em qualquer métrica, inclusive nas suas.

Sala de aula invertida

A maior parte do tempo é vocês trabalhando. Eu puxo o ritmo, sintetizo o que sair das discussões e seguro o tempo. Quem não lê o auto-estudo não consegue responder os quizzes de ativação nem defender em plenário. Isso é sentido.

"Se não mede, está chutando"

inteli.

A frase é velha e aparece em todo workshop de gestão. O problema é o que vem depois: a maioria dos times começa a medir alguma coisa e chuta com mais confiança. Velocity de sprint, número de commits, linhas de código. Métrica boa é a que muda uma decisão. Métrica ruim é a que dá conforto.

Medir atividade

- Quantos commits por dev.
- Quantos story points o time fechou.
- Quantas horas ficaram no código.
- Respondem "o time está ocupado?".

Medir resultado

- Quanto tempo de commit até produção.
- Quantas entregas precisam de rollback.
- Quanto tempo o serviço fica fora do ar.
- Respondem "o time está entregando valor estável?".

```
Atribuído a Peter Drucker, com ressalvas
```

BLOCO 1, 30 MIN

Ativação e síntese

Três quizzes testam o que ficou do auto-estudo. Depois, três mini-sínteses amarram DORA, SLO e observabilidade.

Ativação, 1 de 3, DORA

inteli.

O guia oficial da DORA lista quatro chaves. Qual par abaixo traz apenas métricas DORA?

- Lead Time for Changes e Deployment Frequency
- Velocity de sprint e Cobertura de testes
- Lines of Code e Deployment Frequency
- Tempo de code review e Número de bugs abertos

Ativação, 2 de 3, Métricas que enganam

inteli.

Por que "linhas de código commitadas por dev" é o exemplo clássico de métrica-problema?

- Porque incentiva escrever mais e apagar menos, mesmo quando apagar era o certo.
- Porque dá muito trabalho computacional calcular.
- Porque o valor varia demais entre linguagens diferentes.
- Porque não existe ferramenta confiável que colete.

Ativação, 3 de 3, Três pilares

inteli.

Qual é a diferença essencial entre logs, métricas e traces?

- Logs são texto, métricas são JSON, traces são binário.
- Logs ficam em disco, métricas em banco, traces em cache.
- Logs contam eventos, métricas medem grandezas ao longo do tempo, traces seguem uma requisição entre serviços.
- Logs servem para dev, métricas para produção, traces para DevOps.

As quatro chaves DORA, em uma tela

inteli.

O livro Accelerate, de Nicole Forsgren e a equipe DORA, fechou em quatro indicadores depois de anos olhando times de software do mundo inteiro. Os quatro combinam dois eixos: velocidade e estabilidade. Não é teoria, é o resultado da pesquisa deles.

01

Deployment Frequency

Com que frequência o time libera em produção? Elite faz múltiplas vezes por dia. Baixa performance opera em ciclos mensais.

02

Lead Time for Changes

Quanto tempo leva do commit em `main` até a mudança rodar em produção? Mede o atrito do pipeline inteiro.

03

Change Failure Rate

Qual percentual de entregas exige hotfix, rollback ou correção urgente? Abaixo de 15% é ótimo, acima de 30% é perigoso.

04

Time to Restore Service

Do incidente detectado até o serviço de volta, quanto passa? Elite responde em minutos, baixa performance em dias.

Velocidade e estabilidade juntas

A pesquisa mostra que times de alto desempenho são rápidos e estáveis. Não existe troca, não é um ou outro. Entregar mais e quebrar menos caminham junto quando a engenharia é forte. Frequência de deploy sozinha não diz nada se o CFR está em 40%.

SLI, SLO, SLA, três siglas que não são sinônimos

inteli.

DORA mede o processo de entrega. SLI/SLO/SLA medem o comportamento do serviço rodando. São responsáveis diferentes, documentos diferentes e consequências diferentes. Trocar as siglas vira piada de engenharia.

SLI, indicador

O número que você mede. Ex.: percentual de requisições com latência abaixo de 300 ms na última hora.

SLO, objetivo

A meta interna. Ex.: 99,9% das requisições abaixo de 300 ms no mês. É promessa do time para si mesmo.

SLA, acordo

Contrato com o cliente, com cláusula de crédito ou multa. Sempre mais frouxo que o SLO interno.

Error budget, a ideia do SRE

Se o SLO é 99,9% no mês, você tem 0,1% de falha permitido. Isso vira orçamento para fazer deploy arriscado. Quando o orçamento acaba, o time pausa feature nova e foca em estabilidade até o mês virar. Sem error budget, SLO vira mero discurso.

Três pilares, três perguntas

inteli.

Observabilidade é a capacidade de responder perguntas sobre o sistema sem precisar mexer no código dele. Os três pilares cobrem tipos de pergunta diferentes. O truque é saber qual pilar consultar primeiro em cada situação.

Logs, o que aconteceu

Evento com timestamp e contexto. Bom para debug pontual, post-mortem, rastrear uma requisição específica depois.

Métricas, quanto e quando

Números agregados ao longo do tempo. Bom para dashboards, alertas, ver tendência. A fonte dos SLIs.

Traces, por onde passou

Caminho da requisição entre serviços, com tempo em cada salto. Bom para gargalos em arquitetura distribuída.

Monitoramento não é observabilidade

Monitoramento responde perguntas que você já sabia que ia fazer. A CPU está alta? A fila cresceu? Observabilidade é conseguir responder perguntas que você não sabia que ia fazer. Quando alguém reclama que "só na terça a tela demora", bom monitoramento falha, boa observabilidade salva.

BLOCO 2, 30 MIN

Interpretação em sala

Três FLIPs. Em dupla, vocês leem cenários e métricas, decidem em voz alta, defendem em plenário.

Por que passamos por ativação e síntese?

inteli.

O Bloco 1 não foi só aquecimento. Os quizzes testaram se vocês chegaram na aula com as definições na mão. A síntese amarrou DORA, SLO e observabilidade em um vocabulário comum, porque sem ele a próxima hora vira debate de achismo. Agora vocês entram no Bloco 2 com base firme para **interpretar** painéis reais.

Ativação, quizzes

Cada quiz puxou um conceito do auto-estudo. Se alguém errou, a confusão aparece logo, não lá no exercício final.

Síntese, três telas

DORA, SLI/SLO/SLA e três pilares. Cada um no seu escopo. Não é vocabulário decorativo, vamos usar tudo daqui em diante.

Ponte, interpretação

Saber o que é DORA é fácil. Olhar dois painéis e decidir qual time está melhor é outra coisa. É o que vem agora.

O que muda no Bloco 2

A partir daqui a responsabilidade vira de vocês. Em dupla, vão ler cenários, decidir em voz alta, discordar, defender a escolha. Quem só leu o auto-estudo sem refletir vai sentir no primeiro FLIP. É de propósito.

DORA 01, Deployment Frequency

inteli.

Com que frequência o time coloca código em produção. É o indicador de velocidade mais simples de capturar e o mais fácil de ser mal interpretado se olhado sozinho.

Definição

Número médio de deploys em produção por unidade de tempo (dia, semana, mês). Elite faz várias vezes ao dia. Low performer opera em meses.

No mercado

Amazon ficou famosa por relatar deploys em ambientes internos a cada poucos segundos. Bancos e saúde têm janelas trimestrais por regulação. O comparável é o próprio time, mês a mês.

Como medir

Conte eventos de deploy bem-sucedidos no pipeline de CD. No GitLab, um job de deploy em produção rodado a partir do `main`. Datadog, New Relic e plugins do Jira capturam automaticamente.

Cuidado

Frequência alta sem qualidade vira caos. Só faz sentido acompanhar Deployment Frequency em par com Change Failure Rate. Time que fatia deploy artificialmente para inflar o número está gameando.

DORA 02, Lead Time for Changes

inteli.

O tempo entre o commit entrar no `main` e a mudança estar rodando em produção. Mede o atrito do pipeline inteiro, não só a velocidade de programação.

Definição

Mediana do intervalo entre commit e deploy desse mesmo commit em produção. Elite entrega em menos de 1 dia. Low performer leva de 1 a 6 meses.

No mercado

Times de entrega contínua em fintechs europeias relatam medianas de poucas horas. Empresas com aprovação manual de compliance passam de 30 dias. O alvo depende do contexto, a tendência ao longo de meses é o sinal honesto.

Como medir

Timestamp do commit (Git) menos timestamp do deploy (pipeline de CD). GitHub Actions, GitLab CI e Jenkins expõem isso. LinearB e Swarmia calculam nativamente.

O que o número esconde

Lead Time baixo pode vir de commits muito pequenos ou de testes ausentes. Olhe em conjunto com Change Failure Rate e cobertura de testes. A métrica isolada é enganada por commits cosméticos.

DORA 03, Change Failure Rate

inteli.

Qual porcentagem das entregas exige hotfix, rollback ou correção emergencial. É o contraponto direto de Deployment Frequency, equilibra velocidade com estabilidade.

Definição

Fórmula, deploys com incidente em produção dividido por total de deploys, vezes 100. Elite fica abaixo de 15%. Acima de 45% é sinal de problema sério.

No mercado

Gigantes de streaming e SaaS que publicam dados DORA reportam CFR de dígito único em serviços críticos. Times iniciando medição costumam descobrir CFR acima de 30%. O movimento em 3 meses importa mais que o número bruto.

Como medir

Marque cada deploy que gerou rollback, hotfix ou incidente de severidade alta. Dá para cruzar com PagerDuty, Sentry ou tag de commit de hotfix. Exige disciplina do time em rotular.

Armadilha clássica

Se só o tech lead classifica o que é falha, o número baixa por conveniência. Tenha critério escrito ("qualquer deploy que dispare paging é falha") e revisão cruzada. Sem isso, CFR vira ficção.

DORA 04, Time to Restore Service

inteli.

Do momento em que o incidente aparece até o serviço voltar a funcionar, quanto tempo passa. Mede a capacidade do time de reagir, não a capacidade de evitar que falhe.

Definição

Mediana do intervalo entre detecção do incidente em produção e retorno ao comportamento esperado. Elite fecha em menos de 1 hora. Low performer pode ficar dias.

No mercado

SREs de big techs operam janelas de 15 min em serviços críticos. Empresas de menor maturidade medem em horas. O que separa elite do resto é **runbook** pronto e observabilidade decente.

Como medir

Timestamp de abertura do incidente (PagerDuty, Opsgenie, Jira ITSM) até timestamp de resolução. A tag "post-mortem publicado" ajuda a separar incidente real de alerta barulhento.

Por que essa é a métrica mais honesta

Falha vai acontecer. Medir só prevenção vende ilusão. Time que aceita que falha vai ocorrer e se organiza para reagir rápido tende a ser mais maduro que time que jura que nunca vai falhar. Rollback automático, feature flag e deploy canário nascem aqui.

SALA DE AULA INVERTIDA, 4 MIN EM DUPLA

Qual time está melhor?

Dois painéis DORA de times diferentes. Em dupla, decidam: **qual time está melhor? Por quê?**

Olhem as quatro chaves juntas, não uma de cada vez.

TIME A	
Deployment Frequency	3x por dia
Lead Time for Changes	8 horas
Change Failure Rate	28%
Time to Restore	15 min

TIME B	
Deployment Frequency	1x por mês
Lead Time for Changes	2 semanas
Change Failure Rate	4%
Time to Restore	4 horas

PLENÁRIO Chamo dois pares com escolhas opostas para defender em plenário. **A pegadinha**: Time A é rápido e quebra, Time B é estável e demora

SALA DE AULA INVERTIDA, 4 MIN EM DUPLA

Como gamear a métrica sem agregar valor?

Cada dupla recebe uma métrica. Em quatro minutos, respondam: **como atingir o número sem realmente melhorar o produto?** Seja criativo e honesto.

1, Deployment Frequency

O CEO quer "deploy diário". Você é o tech lead pressionado a mostrar isso no painel até sexta.

`Deploy precisa entregar algo de novo?`

2, Code Review Time

A meta do time é aprovar PRs em menos de 2 horas. Todo mundo está sendo medido por isso.

`Aprovar rápido é ler rápido?`

3, Test Coverage

A política da empresa exige 80% de cobertura. Sem isso, PR não passa no CI.

`Cobrir qualquer linha conta?`

4, Cycle Time do Kanban

O gerente cobra ciclo menor que 3 dias. Todo card antigo aparece em relatório vermelho.

`Fechar ticket é o mesmo que terminar o trabalho?`

PLENÁRIO Cada dupla conta em 30 segundos como gameou. **Se a métrica é fácil de gamear, ela sozinha não serve.** Precisa

SALA DE AULA INVERTIDA, 3 MIN INDIVIDUAL

Qual pilar pega cada problema?

Para cada situação, marquem no caderno: **LOG**, **MÉTRICA**, **TRACE**, ou combinação. Sem consultar. Plenário compara no fim.

1, A tela só demora na terça à tarde

O cliente reclama que a tela de pedidos fica lenta, mas só na segunda metade da terça-feira. Fora disso, tudo normal.

`Quer tendência ao longo do tempo ou evento isolado?`

2, Serviço reiniciando a cada 3 min

Um microsserviço estoura memória, é reiniciado pelo orquestrador, e isso repete ciclicamente desde o deploy da noite passada.

`O padrão aparece bem no gráfico, mas a causa?`

3, Checkout subiu 300ms desde ontem

A latência total do checkout piorou. Não se sabe se é o serviço de pagamento, de estoque, de cálculo de frete ou o banco.

`Precisa seguir a requisição inteira?`

4, Um usuário só reclamou que travou

Um ticket do suporte com o ID da requisição. Só aquele caso, ninguém mais reportou nada parecido.

`Evento pontual precisa de contexto detalhado.`

Depois dos 3 minutos, cada dupla escolhe um cenário e defende sua escolha em voz alta. Nos próximos slides a gente desce cenário por

Resposta Cenário 1, a tela só demora na terça

inteli.

O sintoma é cíclico e só aparece em uma janela da semana. A investigação pede os três pilares em sequência, cada um responde uma pergunta diferente.

1. Métrica

Gráfico de latência p95 no eixo tempo mostra o padrão. Se a terça à tarde aparece como pico consistente, a hipótese está confirmada sem debate.

2. Trace

Pegue uma requisição lenta na janela crítica e veja em qual serviço ou banco ela trava. Isola onde o tempo está sendo gasto.

3. Log

No serviço identificado pelo trace, veja os logs da janela. Costuma aparecer job batch, relatório semanal ou backup competindo por recurso.

Por que os três

Só métrica: você confirma o problema mas não sabe onde. Só log: ruído demais sem filtro temporal. Só trace: acha um caso, não confirma o padrão. A ordem métrica, trace, log economiza tempo em incidente cíclico.

Resposta Cenário 2, serviço reiniciando a cada 3 min

inteli.

O padrão é forte e regular. Métrica confirma em segundos, log revela a causa. Trace ajuda pouco aqui, porque o problema é o próprio processo caindo antes de completar as requisições.

1. Métrica

Memória do pod, número de reinícios por minuto, uso de CPU. Um gráfico denuncia o loop de crash em poucos segundos.

2. Log

Log da aplicação antes do crash traz o stack trace, o OOMKilled ou o panic. Log do orquestrador (Kubernetes events) confirma o motivo do restart.

Trace?

Pouco útil aqui. O processo morre antes de terminar a requisição. Pode ajudar a ver qual endpoint costuma estar em voo quando o crash acontece.

Leitura da pista

"Desde o deploy da noite passada" é dado de ouro. Faça diff entre o commit deployado e o anterior. Combine com métrica de memória para comparar antes e depois. O culpado costuma aparecer em cache crescendo, vazamento de conexão ou loop em estrutura de dados.

Resposta Cenário 3, checkout +300ms desde ontem

inteli.

Esse é o caso de manual de trace distribuído. A latência total subiu, mas o checkout chama pagamento, estoque, frete, antifraude. Sem trace, vira adivinhação entre equipes.

Trace, protagonista

Uma requisição completa de checkout vira um waterfall. Cada serviço aparece com seu tempo. O que aumentou fica visível em um olhar.

Métrica, apoio

Depois de identificar o serviço suspeito, olhe a métrica específica dele (latência de query, tamanho de fila) para confirmar o regime novo.

Log, último recurso

Se o trace mostrou um serviço lento mas não diz por quê, os logs do serviço na janela recente explicam. Cache frio? Query nova? Deploy?

Por que trace primeiro

Arquitetura distribuída sem observabilidade distribuída é caça às bruxas. Já vi time perder 2 dias trocando suposição entre equipes porque ninguém tinha trace, e cada serviço parecia estar bem isoladamente. O trace resolve em minutos.

Resposta Cenário 4, um usuário só reclamou que travou

inteli.

Evento pontual, sem padrão. Métrica agregada não ajuda, a amostra é um caso em milhões. Se o suporte trouxe o ID da requisição, tudo gira em torno de log com contexto.

Log, protagonista

Busque pelo `request_id`, `user_id` ou timestamp exato. Um log estruturado bem montado devolve a história completa daquela execução em segundos.

Trace, se tiver sampling

Se o sistema tem tail sampling para erro, a chance de achar o trace exato existe. Em sampling probabilístico, raramente.

Métrica, não ajuda

Um caso isolado some no agregado. Métrica é para padrão, não para anedota. Se aparecerem muitos casos parecidos, aí vira padrão e volta a ser útil.

Lição estrutural

Para cenários assim, a arquitetura precisa ter **log estruturado com ID de correlação** desde o início. Se o time precisa correr atrás do `request_id` na hora do incidente, já era. É decisão de dia zero, não de crise.

BLOCO 3, 30 MIN

Criação e crítica cruzada

Cada grupo projeta três métricas para o próprio projeto do módulo. Depois, os outros grupos criticam em plenário.

HANDS-ON, EM GRUPO, 15 MIN

Projete três métricas para o seu sistema

1. Em grupo, escolham **três métricas** que vocês querem acompanhar no projeto do módulo. Uma da família DORA, uma de produto ou experiência do usuário, e uma terceira livre.

2. Para cada métrica, escrevam em um cartão (papel ou Miro): **nome, fórmula, fonte de dados, meta inicial, frequência de coleta**.

3. Guardem no bolso: **como alguém poderia gamear essa métrica?** Essa pergunta vai voltar na próxima parte.

4. Escolham um porta-voz. Pitch de 1 minuto em plenário. Cronômetro vai rodar, o professor não prolonga.

CRÍTICA CRUZADA, EM PLENÁRIO, 15 MIN

Pitch de 1 min e crítica obrigatória dos pares

1. Cada grupo apresenta os três cartões em até 1 minuto, **por grupo, não por cartão**. O cronômetro é implacável.
2. Após o pitch, o próximo grupo faz uma de três perguntas obrigatórias, escolhe qual: **"como se gameia?"**, **"o que essa métrica esconde?"**, **"qual efeito colateral?"**.
3. O grupo apresentador **defende** ou **aceita** a crítica. Sem resposta genérica, precisa ser específica para a métrica em discussão.
4. Ao final, cada cartão fica marcado com "aceita" ou "defende". O conjunto vai para o professor como entrega da aula.

Goodhart, Campbell e o risco de medir

inteli.

O exercício que vocês acabaram de fazer não foi truque retórico. Dois pensadores antigos já formalizaram o problema. Ler Goodhart e Campbell te dá linguagem para defender suas escolhas quando um gestor vier pedir "uma métrica única de produtividade".

Lei de Goodhart, 1975

"Quando uma medida se torna meta, ela deixa de ser uma boa medida."

Charles Goodhart, economista britânico. Originalmente sobre política monetária, virou verdade para qualquer sistema em que a métrica vira alvo de pressão.

Campbell's Law, 1976

"Quanto mais uma métrica quantitativa é usada em decisões sociais, mais sujeita ela fica a corrupção."

Donald Campbell, psicólogo. Estudou avaliação de políticas públicas. A ideia se aplica igual quando RH mede engenheiro.

Vanity metrics, o teste rápido

Se a métrica sobe e a vida do usuário (ou do cliente, ou do time) não melhora, é vaidade. Total de downloads sem engajamento, horas trabalhadas, lines of code. Antes de publicar um painel, pergunte: esse número vai mudar alguma decisão? Se não, corte.

Por que isso aparece no TAP da Jacto

inteli.

O TAP da Jacto cita três indicadores de processo que o sistema de vocês precisa monitorar: **cycle time, throughput e backlog health**. Cada um mede uma coisa diferente, e juntos cobrem o fluxo de trabalho do time. Entender o conceito é a parte fácil. Instrumentar no código é o trabalho.

Cycle time

Tempo médio entre iniciar e concluir uma tarefa. Mede o quão rápido algo flui do "to do" ao "done".

Throughput

Quantidade de itens concluídos por período (sprint, semana, dia). Mede volume de saída.

Backlog health

Idade dos itens, variância de tamanho, taxa de crescimento. Indica se o backlog ainda é útil ou virou cemitério.

Da sala para o projeto

Na Sprint 02, vocês vão decidir como coletar, armazenar e expor esses três indicadores. Quem registra cada transição de card? Onde isso vira gráfico? Como o painel vira decisão na retrospectiva? A conversa de hoje te dá o vocabulário. A implementação é com vocês.

Auto-estudo para a Aula 04

inteli.

A Aula 04 é sobre **qualidade de software e IA**. Três provocações antes dela.

Leitura, ISO 25010

Artigo que apresenta as oito famílias de qualidade, manutenibilidade, confiabilidade, performance, segurança, entre outras.

Vídeo, code review com IA

Palestra curta sobre onde o assistente de IA ajuda no code review e onde ele atrapalha. Quando o humano precisa voltar.

Exercício, olhe seu repo

Rode uma análise estática no seu projeto (SonarQube, ESLint com regra strict, Ruff, conforme a linguagem) e guarde o número de warnings.

Entrega da aula de hoje

Os três cartões de métrica do seu grupo, marcados com "aceita" ou "defende" após a crítica cruzada, vão para o AdaLove como evidência do hands-on. Não é entrega de nota, é diagnóstico para a Sprint 02.

inteli.

Até a próxima

Aula 04, Qualidade de software e IA, 07/05